



qrsync v0.3.0

Utility to copy files over WiFi to/from mobile devices inside a terminal using QR codes.

[#tokio](#) [#mobile](#) [#qrcode](#) [#terminal](#)

Follow

[Readme](#)

[14 Versions](#)

[Dependencies](#)

[Dependents](#)

QrSync

BUILD [HTTPS://GITHUB.COM/BADGES/SHIELDS/ISSUES/8671](https://github.com/badges/shields/issues/8671)

BUILD [HTTPS://GITHUB.COM/BADGES/SHIELDS/ISSUES/8671](https://github.com/badges/shields/issues/8671)

CRATES.IO

V0.3.0

DOCS.RS

RUSTDOC

DOWNLOADS

14K

LICENSE

MIT

Utility to copy files over WiFi to/from mobile devices inside a terminal.

- [Install](#)
- [Rust version](#)
- [Platforms support](#)
- [Operational modes](#)
- [IPv6 support](#)
- [Command line options](#)
- [Acknowledgement](#)
- [License](#)

When I built QrSync, it was only meant to send files from a terminal to a mobile device, then I found the amazing [grcp](#) and I took some ideas from it and implemented also the possibility to copy file from the mobile device to the computer running QrSync.

Install

Github Actions releases [binaries](#) for various architectures when a new tag is pushed:

- x84-64 Linux GNU
- x86-64 Darwin
- aarch64 Linux GNU
- armv7 Linux GNU

Alternatively you can install the latest tag directly from [crates.io](#):

```
>>> cargo install qrsync
```

Rust version

QrSync builds against stable Rust >= 1.60.

Platforms support

QrSync has been tested on Linux and MacOSX.

It currently also build against Windows, but it has not being tested. On *nix it uses [pnet](#) to auto discover the primary interface and its IP address and bind against it. Pnet have a some complex dependencies to build against Windows (see [here](#) for more info), so on this platform QrSync makes the `--ip-address` command-line option mandatory and `pnet` is not built at all.

Operational modes

QrSync can run in two modes, depending on command line options:

- **Send mode:** this mode is selected when a file is passed to the command line. QrSync will generate a QR code on the terminal and start the HTTP server in send mode. Example:

```
>>> qrsync my_document.pdf
INFO qrsync::http > Send mode enabled for file /home/bigo/my_document.pdf
INFO qrsync::http > Scan this QR code with a QR code reader app to open the URL http://192.168.1.11:5566/Q2FyZ28ud
```

- **Receive mode:** this mode is selected if no file is passed to the command line. QrSync will generate a QR code on the terminal and start the HTTP server in receive mode from the current folder. A specific folder to save received files can be specified with `--root-dir` command line option. Example:

```
>>> qrsync
INFO qrsync::http > Receive mode enabled inside directory /home/bigo
INFO qrsync::http > Scan this QR code with a QR code reader app to open the URL http://192.168.1.11:5566/receive
```

IPv6 support

QrSync tries to guess which interface to use and which address to bind on the selected interface. In case you want to use IPv6, ensure you have a valid non link-local address and specify `--ipv6` command line argument. Remember, the IP address can be always overridden using `--ip-address` command line argument.

Command line options

USAGE:

```
qrsync [FLAGS] [OPTIONS] [filename]
```

ARGS:

```
<filename>    File to be send to the mobile device
```

FLAGS:

```
-d, --debug      Enable QrSync debug
-h, --help       Prints help information
-6, --ipv6       Prefer IPv6 over IPv4
-l, --light-term  Draw QR in a terminal with light background
-V, --version     Prints version information
```

OPTIONS:

```
-i, --ip-address <ip-address>  IP address to bind the HTTP server to. Default to primary interface
-p, --port <port>              Port to bind the HTTP server to [default: 5566]
-r, --root-dir <root-dir>      Root directory to store files in receive mode
```

Acknowledgement

- [qrcp](#): I took many ideas from this amazing project and "stole" most of the HTML Bootstrap based UI.
- [axum](#): A great HTTP framework for Rust, very expandable and simple to use.
- [qr2term](#): Terminal based QR rendering library.
- [clap](#): Oh man, where do I start telling how much do I love Clap?

License

See [LICENSE](#) file.

Metadata

-  over 1 year ago
-  v1.60.0
-  MIT
-  139 KiB

Install

```
cargo install qrsync
```

Running the above command will globally install the **qrsync** binary.

Install as library

Run the following Cargo command in your project directory:

```
cargo add qrsync
```

Or add the following line to your Cargo.toml:

```
qrsync = "0.3.0"
```

Documentation

 docs.rs/qrsync/0.3.0

Repository

 github.com/crisidev/qrsync

Owners

 Matteo Bigoi

Categories

- [Command line utilities](#)
- [Multimedia](#)

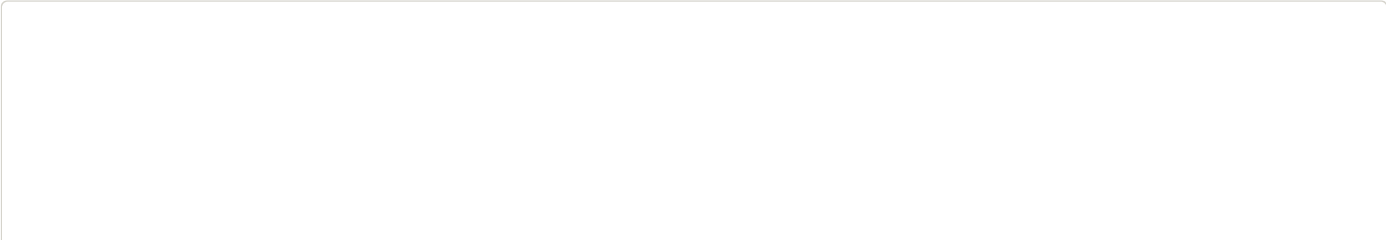
[Report crate](#)

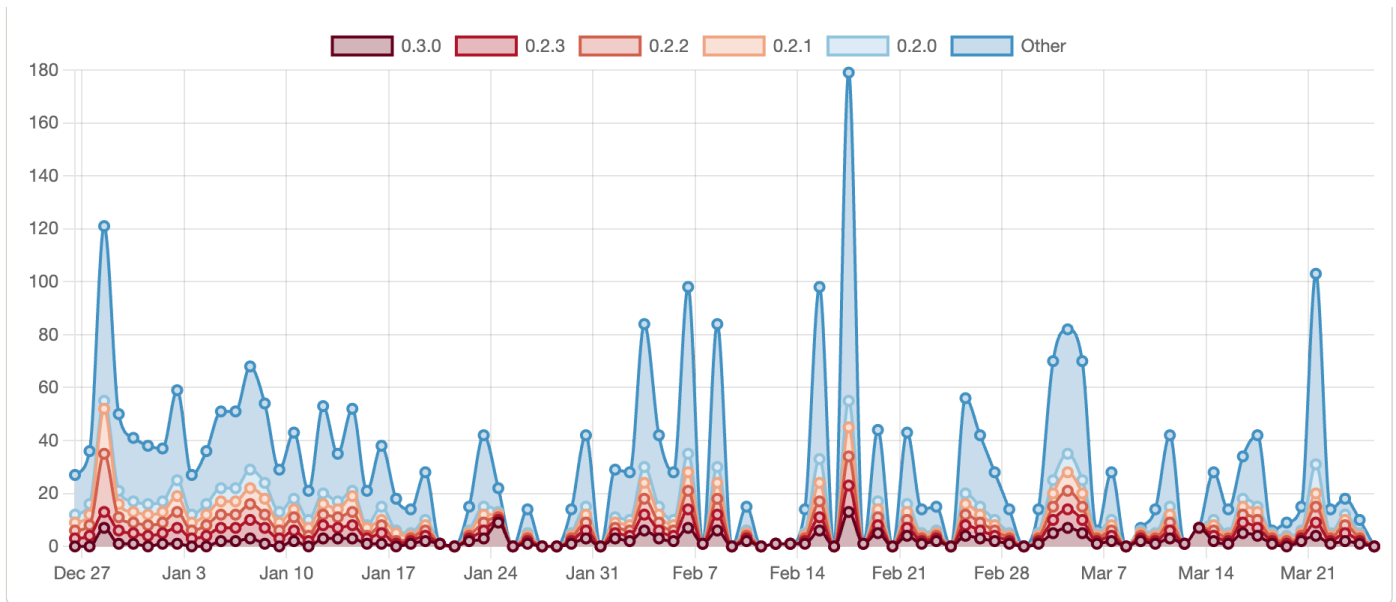
Stats Overview

 14,299 Downloads all time	 14 Versions published
---	---

Downloads over the last 90 days

Display as Stacked ▾





Rust

rust-lang.org

[Rust Foundation](#)

[The crates.io team](#)

Get Help

[The Cargo Book](#)

[Support](#)

[System Status](#)

[Report a bug](#)

Policies

[Usage Policy](#)

[Security](#)

[Privacy Policy](#)

[Code of Conduct](#)

[Data Access](#)

Social

rust-lang/crates.io

[#t-crates-io](#)

[@cratesiostatus](#)